Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №2
на тему
«СИММЕТРИЧНАЯ КРИПТОГРАФИЯ. СТБ 34.101.31-2011»
БГУИР 6-05-0612-02 21

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

(дата, подпись студента)

Проверил ассистент каф. информатики
ПОЛЯКОВ Кирилл Борисович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Теоретические сведения 4
2.1 Стандарт СТБ 34.101.31-2011 4
2.2 Подстановка H и преобразование G 4
2.3 Такт шифрования 5
2.4 Расписание тактовых ключей 5
2.5 Режим простой замены 6
2.6 Режим гаммирования с обратной связью 6
3 Выполнение работы 7
3.1 Структура программного средства 7
3.2 Реализация блочного шифра 7
3.3 Реализация режима простой замены 7
3.4 Реализация режима гаммирования с обратной связью 8
3.5 Тестирование 8
Заключение 9
Список использованных источников 10
Приложение А (Обязательное) Листинг программного кода 11

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является изучение стандарта шифрования СТБ 34.101.31-2011 и получение практических навыков работы с блочными шифрами путём самостоятельной реализации базового алгоритма шифрования блока и двух режимов его применения: простой замены (*ECB*) и гаммирования с обратной связью (*CFB*).

Для достижения поставленной цели необходимо решить следующие задачи:

1 Изучить теоретические основы стандарта СТБ 34.101.31-2011: структуру блочного шифра *belt*, подстановку *H*, преобразование *G* и расписание тактовых ключей.

2 Реализовать базовый блочный шифр для блока размером 128 бит и ключа размером 256 бит с восемью тактами шифрования.

3 Реализовать два режима работы блочного шифра: простой замены и гаммирования с обратной связью.

4 Провести тестирование корректности реализации на официальных тестовых примерах приложения А стандарта и посредством последовательного шифрования и дешифрования.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Стандарт СТБ 34.101.31-2011

СТБ 34.101.31-2011 – государственный стандарт Республики Беларусь, устанавливающий семейство криптографических алгоритмов, предназначенных для обеспечения конфиденциальности и контроля целостности данных [1]. Стандарт определяет восемь групп алгоритмов: шифрование в режимах простой замены, сцепления блоков, гаммирования с обратной связью и счётчика, выработку имитовставки, одновременное шифрование и имитозащиту данных и ключа, а также хеширование. Первые четыре группы обеспечивают конфиденциальность, пятая – контроль целостности.

Базовым элементом всех режимов является блочный шифр *belt*. Его основные параметры приведены ниже.

– Размер блока – 128 бит (16 байт).
– Размер ключа – 256 бит (32 байта).
– Число тактов шифрования – 8.
– Размер тактового ключа – 32 бита.

В отличие от классической сети Фейстеля, применяемой, например, в ГОСТ 28147-89, шифр *belt* построен по схеме подстановок-перестановок и оперирует четырьмя 32-битными словами a, b, c, d, которые рассматриваются как единое состояние. Редакция стандарта 2020 года перенесла криптографические алгоритмы без функциональных изменений, поэтому приведённое далее описание соответствует обеим редакциям [2]. Полный текст стандарта доступен на сайте лаборатории [3].

2.2 Подстановка H и преобразование G

Подстановка H представляет собой фиксированную таблицу из 256 байт, которая ставит в соответствие 8-битной входной строке 8-битную выходную. Она задана стандартом и не зависит от ключа.

На 32-битное слово подстановка действует побайтово. Если слово u представлено байтами u_0, u_1, u_2, u_3, где u_0 – младший байт, то

$$H(u) = H(u_0) + 2^8 H(u_1) + 2^{16} H(u_2) + 2^{24} H(u_3). \tag{2.1}$$

Преобразование G_r вычисляет $H(u)$ и выполняет циклический сдвиг результата влево на r бит:

$$G_r(u) = \mathrm{RotHi}_r(H(u)). \tag{2.2}$$

В тактах шифрования используются значения $r = 5$, $r = 13$ и $r = 21$.

2.3 Такт шифрования

Шифрование блока на ключе выполняется за восемь тактов. На каждом такте к словам a, b, c, d применяется последовательность операций, в которой k_j обозначает очередной тактовый ключ, а i – номер такта:

$$\begin{aligned}
b &\leftarrow b \oplus G_5(a \boxplus k_{7i-6}),\\
c &\leftarrow c \oplus G_{21}(d \boxplus k_{7i-5}),\\
a &\leftarrow a \boxminus G_{13}(b \boxplus k_{7i-4}),\\
e &\leftarrow G_{21}(b \boxplus c \boxplus k_{7i-3}) \oplus i,\\
b &\leftarrow b \boxplus e,\\
c &\leftarrow c \boxminus e,\\
d &\leftarrow d \boxplus G_{13}(c \boxplus k_{7i-2}),\\
b &\leftarrow b \oplus G_{21}(a \boxplus k_{7i-1}),\\
c &\leftarrow c \oplus G_5(d \boxplus k_{7i}).
\end{aligned} \tag{2.3}$$

Здесь символы $\boxplus$ и $\boxminus$ обозначают сложение и вычитание по модулю 2^{32}, а $\oplus$ – побитовое сложение по модулю 2.

По окончании такта регистры переставляются обменами $a \leftrightarrow b$, $c \leftrightarrow d$, $b \leftrightarrow c$. После восьмого такта выходной блок образуется склеиванием регистров в порядке $bdac$:

$$Y = b \,||\, d \,||\, a \,||\, c. \tag{2.4}$$

Особенность шифра *belt* состоит в том, что расшифрование использует те же операции, что и зашифрование, но тактовые ключи применяются в обратном порядке, а перестановка в конце такта имеет вид $a \leftrightarrow b$, $c \leftrightarrow d$, $a \leftrightarrow d$. Выход расшифрования образуется склеиванием регистров в порядке $cadb$:

$$X = c \,||\, a \,||\, d \,||\, b. \tag{2.5}$$

2.4 Расписание тактовых ключей

Ключ θ разбивается на восемь 32-битных слов $\theta_1, \theta_2, \ldots, \theta_8$. Слово считается записанным младшим байтом вперёд. Тактовые ключи k_j образуются по правилу

$$k_j = \theta_{(j-1) \bmod 8} + 1. \tag{2.6}$$

При зашифровании такт с номером i использует ключи $k_{7i-6}, k_{7i-5}, \ldots, k_{7i}$, а при расшифровании – те же ключи в обратном порядке: $k_{7i}, k_{7i-1}, \ldots, k_{7i-6}$.

2.5 Режим простой замены

В режиме простой замены сообщение разбивается на блоки по 128 бит, каждый из которых шифруется независимо от остальных на одном и том же ключе. Режим применяется к сообщениям, содержащим хотя бы один полный блок.

Если длина сообщения не кратна 128 битам, последний неполный блок обрабатывается процедурой сдвига. Пусть сообщение содержит n полных блоков $P_1, \ldots, P_n$ и неполный блок P_{n+1} длины r. Тогда шифртекст имеет вид

$$Y = C_1 \,||\, \ldots \,||\, C_{n-1} \,||\, E\big(P_{n+1} \,||\, C_{n[r:16]}\big) \,||\, C_{n[0:r]}, \tag{2.7}$$

где E – зашифрование блока. Таким образом, длина шифртекста равна длине сообщения, а дополнение по схеме *PKCS#7* не применяется.

Преимуществом режима является возможность независимой обработки блоков, а недостатком то, что одинаковые блоки данных всегда преобразуются в одинаковые блоки шифртекста.

2.6 Режим гаммирования с обратной связью

В режиме гаммирования с обратной связью каждой части сообщения ставится в соответствие часть шифртекста той же длины: часть сообщения складывается по модулю 2 с результатом зашифрования предыдущего блока шифртекста. Входными данными являются сообщение X произвольной длины, ключ θ и синхропосылка S длиной 128 бит. Сообщение разбивается на части $X_1, X_2, \ldots, X_n$, из которых все, кроме последней, имеют длину 128 бит.

Зашифрование выполняется по правилам

$$Y_0 = S, \quad Y_i = X_i \oplus L_{|X_i|}(F(Y_{i-1})), \quad i = 1, 2, \ldots, n, \tag{2.8}$$

где F – зашифрование блока, а L_s обозначает выборку s левых бит аргумента. Расшифрование отличается лишь тем, что обратная связь берётся от принятого шифртекста:

$$X_0 = S, \quad Y_i = X_i \oplus L_{|X_i|}(F(X_{i-1})). \tag{2.9}$$

Поскольку длина последней части X_n может быть меньше 128 бит, режим применим к сообщениям произвольной длины без дополнения. Синхропосылка не является секретной и передаётся получателю вместе с шифртекстом. Каждый блок шифртекста зависит от всех предыдущих, что устраняет основной недостаток режима простой замены.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программного средства

Реализация разделена по видам преобразования: базовый блочный шифр вынесен в отдельный модуль, а оба режима его применения используют общие функции *BlockEncrypt* и *BlockDecrypt*. Модули расположены в пакете *crypto/belt* и включают:

– *belt.go* – базовый блочный шифр: подстановка H, преобразование G_r, такт шифрования, расписание тактовых ключей, шифрование и расшифрование блока;

– *ecb.go* – режим простой замены, включая процедуру сдвига для неполного блока;

– *gammaFeedback.go* – режим гаммирования с обратной связью;

Точками входа являются *BlockEncrypt* и *BlockDecrypt* для одиночного блока, *ECBEncrypt* и *ECBDecrypt* для режима простой замены, *GammaWithFeedbackEncrypt* и *GammaWithFeedbackDecrypt* для режима гаммирования с обратной связью. Функции проверяют длину ключа, равную 32 байтам, а функция зашифрования в режиме гаммирования вырабатывает случайную синхропосылку и возвращает её вместе с шифртекстом.

3.2 Реализация блочного шифра

Подстановка H хранится в виде шестнадцатеричной строки из 256 байт и преобразуется в массив при инициализации пакета. Функция *hWord* применяет подстановку побайтово, а функция g выполняет циклический сдвиг влево с помощью стандартной функции *bits.RotateLeft32*.

Ключ разбивается на восемь 32-битных слов функцией *keyWords*. Функция *subkeysForRound* формирует семь тактовых ключей для такта с номером i: при зашифровании в прямом порядке $k_{7i-6}…k_{7i}$, при расшифровании – в обратном. Единая функция *beltRound* реализует девять операций такта, одинаковых для обоих направлений.

Функции *BlockEncrypt* и *BlockDecrypt* выполняют восемь тактов, каждый из которых заканчивается перестановкой регистров, и собирают результат из четырёх слов в порядке их следования. Перестановки после первых семи тактов заданы порядком чтения регистров в следующем такте, а перестановка после восьмого такта, когда следующего такта уже нет, выполняется явными обменами в конце функции. Результат работы этих функций проверен на официальных примерах шифрования и дешифрования блока.

3.3 Реализация режима простой замены

Режим *ECB* реализован функциями *ECBEncrypt* и *ECBDecrypt*. При длине сообщения, кратной размеру блока, каждый блок шифруется независимо. Если длина не кратна 128 битам, вызываются функции

ecbEncryptLastPartial и *ecbDecryptLastPartial*, реализующие процедуру сдвига: неполный блок дополняется хвостом предыдущего блока шифртекста, дополненный блок зашифровывается, а на место неполного блока помещаются первые r байт этого шифртекста. Функция зашифрования требует наличия хотя бы одного полного блока, что соответствует требованиям стандарта.

3.4 Реализация режима гаммирования с обратной связью

Режим *CFB* реализован функциями *GammaWithFeedbackEncrypt* и *GammaWithFeedbackDecrypt*. Функция зашифрования вырабатывает случайную 128-битную синхропосылку с помощью генератора *crypto/rand* и возвращает её вместе с шифртекстом. Для каждой части сообщения зашифруется предыдущий блок шифртекста, и полученный блок складывается по модулю 2 с очередной частью открытого текста. Для последней неполной части используются только левые $|X_n|$ бит результата зашифрования, что обеспечивает работу с сообщениями произвольной длины без дополнения. Функция расшифрования в качестве обратной связи использует принятый шифртекст.

3.5 Тестирование

Корректность реализации проверялась на официальных тестовых примерах приложения А стандарта, а также посредством последовательного шифрования и дешифрования. Тестовые данные стандарта построены на основе самой подстановки H: открытые тексты берутся из её начала, ключи – из области $H[128 : 160]$, синхропосылки – из области $H[192 : 208]$.

Все примеры, включая шифрование и дешифрование блока (А.1, А.4), режим простой замены для сообщений длиной 48 и 47 байт (А.9-1, А.9-2) и режим гаммирования с обратной связью (А.13), совпали с эталонными значениями. Дополнительно проведены круговые тесты: для режима простой замены при длинах от 16 до 48 байт, для режима гаммирования с обратной связью – от 1 до 48 байт. Во всех случаях открытый текст полностью восстанавливается.

Корректность реализации проверялась двумя независимыми способами: на официальных тестовых примерах приложения А стандарта и посредством круговых тестов (последовательного шифрования и дешифрования). Эти способы не взаимозаменяемы. Круговой тест подтверждает лишь согласованность шифрования и дешифрования между собой: если общий код такта *beltRound*, используемый и при шифровании, и при расшифровании, содержит неточность, обе функции ошибаются согласованно, расшифрование по-прежнему отменяет шифрование — но результат не соответствует стандарту. Проверка по официальным примерам отвергает такую реализацию, поэтому единство двух подходов даёт существенно более строгую гарантию, чем любой из них по отдельности.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы реализован блочный шифр *belt* стандарта СТБ 34.101.31-2011 для блока 128 бит и ключа 256 бит с поддержкой двух режимов работы: простой замены (*ECB*) и гаммирования с обратной связью (*CFB*).

Реализация базового блочного шифра включает подстановку H из 256 байт, преобразование G_r с циклическим сдвигом, восемь тактов с циклической перестановкой регистров и расписание тактовых ключей с прямым и обратным порядком применения. Реализация режима простой замены учитывает процедуру сдвига для неполного последнего блока, определённую стандартом, благодаря чему не требуется дополнение по схеме *PKCS#7*. Реализация режима гаммирования с обратной связью обеспечивает обработку сообщений произвольной длины без дополнения и зависимость каждого блока шифртекста от всех предыдущих.

Корректность подтверждена на официальных тестовых примерах приложения А стандарта и круговыми тестами. Получены практические навыки реализации блочных шифров и их режимов работы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Информационные технологии. Защита информации. Криптографические алгоритмы шифрования и контроля целостности : СТБ 34.101.31-2011. – Введ. 01.06.2012. – Минск: Госстандарт, 2011. – 23 с.

[2] Информационные технологии и безопасность. Алгоритмы шифрования и контроля целостности : СТБ 34.101.31-2020. – Введ. 01.09.2021. – Минск: Госстандарт, 2020. – 40 с.

[3] СТБ 34.101.31-2011. Криптографические алгоритмы шифрования и контроля целостности (полный текст) [Электронный ресурс]. – Режим доступа: https://apmi.bsu.by/assets/files/std/belt-spec27.pdf. – Дата доступа: 15.09.2026.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Базовый блочный шифр

```
package belt

import (
  "encoding/binary"
  "encoding/hex"
  "fmt"
  "math/bits"
)

const (
  BlockSize = 16
  KeySize   = 32
)

// H substitution
var hTable = mustDecodeH(
  "b194bac80a08f53b366d008e584a5de4" +
    "8504fa9d1bb6c7ac252e72c202fdce0d" +
    "5be3d61217b96181fe6786ad716b890b" +
    "5cb0c0ff33c356b835c405aed8e07f99" +
    "e12bdc1ae28257ec703fccf095ee8df1" +
    "c1ab76389fe678caf7c6f860d5bb9c4f" +
    "f33c657b637c306add4ea7799eb23d31" +
    "3e98b56e27d3bccf591e181f4c5ab793" +
    "e9dee72c8f0c0fa62ddb49f46f739647" +
    "06075316ed247a3739cba38303a98bf6" +
    "92bd9b1ce5d141015445fbc95e4d0ef2" +
    "682080aa227d642f2687f93490405511" +
    "be32971343fc9a48a02a885f194b09a1" +
    "7ecda4d01544af8ca58450bf66d2e88a" +
    "a2d7465242a8dfb36974c551eb232921" +
    "d4efd9b43a622875911410ea776cda1d",
)

func mustDecodeH(s string) [256]byte {
  var table [256]byte

  raw, err := hex.DecodeString(s)
  if err != nil {
    panic("belt: invalid H table: " + err.Error())
  }
  copy(table[:], raw)

  return table
}

// hWord applies H substitution to each byte of 32-bits word
func hWord(u uint32) uint32 {
  return uint32(hTable[byte(u)]) |
    uint32(hTable[byte(u>>8)])<<8 |
    uint32(hTable[byte(u>>16)])<<16 |
    uint32(hTable[byte(u>>24)])<<24
}

func g(u uint32, r int) uint32 {
  return bits.RotateLeft32(hWord(u), r)
```

```
}

// keyWords divides key into 8 32-bit words
func keyWords(key []byte) [8]uint32 {
  var theta [8]uint32
  for i := range theta {
    theta[i] = binary.LittleEndian.Uint32(key[i*4 : i*4+4])
  }

  return theta
}

// K[j] = theta[(j-1) mod 8].
func subkeysForRound(theta [8]uint32, i int, encrypt bool) [7]uint32 {
  var subkeys [7]uint32
  for j := range subkeys {
    if encrypt {
      subkeys[j] = theta[(7*i-7+j)%8]
    } else {
      subkeys[j] = theta[(7*i-1-j)%8]
    }
  }

  return subkeys
}

// beltRound выполняет один такт шифрования/расшифрования над словами
// a, b, c, d. Формулы такта одинаковы для обоих направлений и различаются
// только порядком тактовых ключей.
func beltRound(a, b, c, d uint32, i uint32, k [7]uint32) (uint32, uint32, uint32,
uint32) {
  b ^= g(a+k[0], 5)
  c ^= g(d+k[1], 21)
  a -= g(b+k[2], 13)

  e := g(b+c+k[3], 21) ^ i

  b += e
  c -= e
  d += g(c+k[4], 13)
  b ^= g(a+k[5], 21)
  c ^= g(d+k[6], 5)

  return a, b, c, d
}

func BlockEncrypt(src, key []byte) []byte {
  theta := keyWords(key)

  a := binary.LittleEndian.Uint32(src[0:4])
  b := binary.LittleEndian.Uint32(src[4:8])
  c := binary.LittleEndian.Uint32(src[8:12])
  d := binary.LittleEndian.Uint32(src[12:16])

  // Такты с циклической перестановкой регистров abcd -> bdac -> dcba -> cadb.
  a, b, c, d = beltRound(a, b, c, d, 1, subkeysForRound(theta, 1, true))
  b, d, a, c = beltRound(b, d, a, c, 2, subkeysForRound(theta, 2, true))
  d, c, b, a = beltRound(d, c, b, a, 3, subkeysForRound(theta, 3, true))
  c, a, d, b = beltRound(c, a, d, b, 4, subkeysForRound(theta, 4, true))
  a, b, c, d = beltRound(a, b, c, d, 5, subkeysForRound(theta, 5, true))
  b, d, a, c = beltRound(b, d, a, c, 6, subkeysForRound(theta, 6, true))
  d, c, b, a = beltRound(d, c, b, a, 7, subkeysForRound(theta, 7, true))
  c, a, d, b = beltRound(c, a, d, b, 8, subkeysForRound(theta, 8, true))
```

```
  // abcd -> bdac (a<->b, c<->d, b<->c).
  a, b = b, a
  c, d = d, c
  b, c = c, b

  dst := make([]byte, BlockSize)
  binary.LittleEndian.PutUint32(dst[0:4], a)
  binary.LittleEndian.PutUint32(dst[4:8], b)
  binary.LittleEndian.PutUint32(dst[8:12], c)
  binary.LittleEndian.PutUint32(dst[12:16], d)

  return dst
}

func BlockDecrypt(src, key []byte) []byte {
  theta := keyWords(key)

  a := binary.LittleEndian.Uint32(src[0:4])
  b := binary.LittleEndian.Uint32(src[4:8])
  c := binary.LittleEndian.Uint32(src[8:12])
  d := binary.LittleEndian.Uint32(src[12:16])

  a, b, c, d = beltRound(a, b, c, d, 8, subkeysForRound(theta, 8, false))
  c, a, d, b = beltRound(c, a, d, b, 7, subkeysForRound(theta, 7, false))
  d, c, b, a = beltRound(d, c, b, a, 6, subkeysForRound(theta, 6, false))
  b, d, a, c = beltRound(b, d, a, c, 5, subkeysForRound(theta, 5, false))
  a, b, c, d = beltRound(a, b, c, d, 4, subkeysForRound(theta, 4, false))
  c, a, d, b = beltRound(c, a, d, b, 3, subkeysForRound(theta, 3, false))
  d, c, b, a = beltRound(d, c, b, a, 2, subkeysForRound(theta, 2, false))
  b, d, a, c = beltRound(b, d, a, c, 1, subkeysForRound(theta, 1, false))

  // abcd -> cadb (инверсии a<->b, c<->d, a<->d).
  a, b = b, a
  c, d = d, c
  a, d = d, a

  dst := make([]byte, BlockSize)
  binary.LittleEndian.PutUint32(dst[0:4], a)
  binary.LittleEndian.PutUint32(dst[4:8], b)
  binary.LittleEndian.PutUint32(dst[8:12], c)
  binary.LittleEndian.PutUint32(dst[12:16], d)

  return dst
}

func validateKey(key []byte) error {
  if len(key) != KeySize {
    return fmt.Errorf("belt: key must be %d bytes, got %d", KeySize, len(key))
  }

  return nil
}
```

Листинг А.2 – Режим простой замены

```
package belt

import "fmt"

func ECBEncrypt(data, key []byte) ([]byte, error) {
  if err := validateKey(key); err != nil {
    return nil, err
```

```
  }
  if len(data) < BlockSize {
    return nil, fmt.Errorf(
     "belt-ecb: message must contain at least one %d-byte block, got %d bytes",
      BlockSize, len(data),
    )
  }

  if len(data)%BlockSize == 0 {
    result := make([]byte, len(data))
    for i := 0; i < len(data); i += BlockSize {
      copy(result[i:i+BlockSize], BlockEncrypt(data[i:i+BlockSize], key))
    }
    return result, nil
  }

  return ecbEncryptLastPartial(data, key), nil
}

func ECBDecrypt(data, key []byte) ([]byte, error) {
  if err := validateKey(key); err != nil {
    return nil, err
  }
  if len(data) < BlockSize {
    return nil, fmt.Errorf(
        "belt-ecb: ciphertext must contain at least one %d-byte block, got
%d bytes",
      BlockSize, len(data),
    )
  }

  if len(data)%BlockSize == 0 {
    result := make([]byte, len(data))
    for i := 0; i < len(data); i += BlockSize {
      copy(result[i:i+BlockSize], BlockDecrypt(data[i:i+BlockSize], key))
    }
    return result, nil
  }

  return ecbDecryptLastPartial(data, key), nil
}

func ecbEncryptLastPartial(data, key []byte) []byte {
  n := len(data) / BlockSize
  rem := len(data) % BlockSize
  result := make([]byte, len(data))

  for i := 0; i < n-1; i++ {
    copy(result[i*BlockSize:], BlockEncrypt(data[i*BlockSize:(i+1)*BlockSize],
key))
  }

  lastCipher := BlockEncrypt(data[(n-1)*BlockSize:n*BlockSize], key)

  augmented := make([]byte, BlockSize)
  copy(augmented, data[n*BlockSize:])
  copy(augmented[rem:], lastCipher[rem:])

  copy(result[(n-1)*BlockSize:n*BlockSize], BlockEncrypt(augmented, key))
  copy(result[n*BlockSize:], lastCipher[:rem])

  return result
}
```

```
func ecbDecryptLastPartial(data, key []byte) []byte {
  n := len(data) / BlockSize
  rem := len(data) % BlockSize
  result := make([]byte, len(data))

  for i := 0; i < n-1; i++ {
    copy(result[i*BlockSize:], BlockDecrypt(data[i*BlockSize:(i+1)*BlockSize],
key))
  }

  cAug := data[(n-1)*BlockSize : n*BlockSize]
  tail := data[n*BlockSize:]
  augmented := BlockDecrypt(cAug, key)

  lastCipher := make([]byte, BlockSize)
  copy(lastCipher, tail)
  copy(lastCipher[rem:], augmented[rem:])

  copy(result[(n-1)*BlockSize:n*BlockSize], BlockDecrypt(lastCipher, key))
  copy(result[n*BlockSize:], augmented[:rem])

  return result
}
```

Листинг А.3 – Режим гаммирования с обратной связью

```
package belt

import (
  "crypto/rand"
  "errors"
  "fmt"
)

func GammaWithFeedbackEncrypt(data, key []byte) ([]byte, []byte, error) {
  if err := validateKey(key); err != nil {
    return nil, nil, err
  }
  if len(data) == 0 {
    return nil, nil, errors.New("belt-cfb: message is empty")
  }

  sync := make([]byte, BlockSize)
  if _, err := rand.Read(sync); err != nil {
    return nil, nil, fmt.Errorf("belt-cfb: generate sync: %w", err)
  }

  return gammaWithFeedbackEncrypt(data, key, sync), sync, nil
}

func gammaWithFeedbackEncrypt(data, key, sync []byte) []byte {
  cipherText := make([]byte, len(data))
  yPrev := sync

  for offset := 0; offset < len(data); offset += BlockSize {
    end := min(offset+BlockSize, len(data))

    yCurr := BlockEncrypt(yPrev, key)
    for i := offset; i < end; i++ {
      cipherText[i] = data[i] ^ yCurr[i-offset]
    }
```

```
    yPrev = cipherText[offset:end]
  }

  return cipherText
}

func GammaWithFeedbackDecrypt(data, key, sync []byte) ([]byte, error) {
  if err := validateKey(key); err != nil {
    return nil, err
  }
  if len(sync) != BlockSize {
   return nil, fmt.Errorf("belt-cfb: sync must be %d bytes, got %d", BlockSize,
len(sync))
  }
  if len(data) == 0 {
    return nil, errors.New("belt-cfb: ciphertext is empty")
  }

  return gammaWithFeedbackDecrypt(data, key, sync), nil
}

func gammaWithFeedbackDecrypt(data, key, sync []byte) []byte {
  plainText := make([]byte, len(data))
  xPrev := sync

  for offset := 0; offset < len(data); offset += BlockSize {
    end := min(offset+BlockSize, len(data))

    xCurr := BlockEncrypt(xPrev, key)
    for i := offset; i < end; i++ {
      plainText[i] = data[i] ^ xCurr[i-offset]
    }

    xPrev = data[offset:end]
  }

  return plainText
}
```